



Welcome to Day 4

CSS Layouts & Flexbox

Today we dive into one of the most powerful areas of front-end development — controlling how elements are positioned and laid out on the page. By the end of this session, you'll be building real navbars and card layouts using Flexbox like a pro.

Today's Topics

- Display Property & Block vs Inline
- CSS Positioning
- Flexbox: Direction, Justify, Align, Gap
- Navbar & Card Layout Design

Practical Tasks

- Build a Navigation Bar
- Create a 3-Card Layout
- Design a Responsive Section

📌 "Layouts help organize webpage content professionally."

The Display Property

The `display` property is the foundation of CSS layout. It controls how an element is rendered in the document flow — whether it takes up the full width, sits inline with text, or becomes a flex container.

block

Takes up the full available width. Always starts on a new line. Examples: `div`, `p`, `h1`

inline

Only takes as much width as needed. Stays in the same line as surrounding content. Examples: `span`, `a`

inline-block

Combines both: stays in line but allows you to set width and height.

flex

Turns the element into a flex container, enabling powerful alignment and layout control for its children.

none

Hides the element completely — it takes up no space in the layout at all.

Example

```
div {  
  display: block;  
}
```

Result

Each `div` appears on its own line, stacked vertically one after the other — the default browser behavior.

Block vs Inline Elements

Understanding the difference between block and inline elements is essential for controlling page structure. Each type behaves very differently in the document flow.

Block Elements

Always start on a new line and stretch to fill the full container width.

- `div` — generic container
- `h1`–`h6` — headings
- `p` — paragraphs

```
<div>Block 1</div>
<div>Block 2</div>
<!-- Each on its own line -->
```

Inline Elements

Flow with the surrounding text. Only take up as much space as their content requires.

- `span` — generic inline wrapper
- `a` — anchor/link
- `strong` — bold text

```
<span>Hello</span>
<span>Students</span>
<!-- Both on the same line -->
```

Inline-Block

The best of both worlds — elements stay on the same line *and* accept custom width/height values.

```
display: inline-block;
```

CSS Positioning

The `position` property lets you take elements out of the normal document flow and place them exactly where you want. This is key for creating overlays, sticky headers, and centered banners.

1	static Default positioning. Elements appear in the normal document flow. Top/left/right/bottom have no effect.
2	relative Positioned relative to its own normal position. Acts as a reference point for absolutely positioned children.
3	absolute Removed from normal flow. Positioned relative to the nearest <code>relative</code> ancestor.
4	fixed Stays fixed relative to the browser window — perfect for sticky navbars that don't scroll away.
5	sticky Behaves like relative until it hits a scroll threshold, then behaves like fixed. Great for section headers.

Real-World Example: Centered Banner Title

```
.banner {
  background-image: url("bg.jpg");
  height: 400px;
  background-size: cover;
  position: relative; /* Reference point for child */
}
.banner h1 {
  position: absolute;
  top: 50%; left: 50%;
  transform: translate(-50%, -50%); /* Perfect centering */
  color: white; font-size: 40px;
}
```

Introduction to Flexbox

Flexbox (Flexible Box Layout) is a CSS layout model that makes it easy to align and distribute space among items in a container — even when their sizes are unknown or dynamic. It's one of the most important tools in a front-end developer's toolkit.

How to Enable Flexbox

Just one line on the parent container transforms it into a flex container:

```
.container {  
  display: flex;  
}
```

All direct children become **flex items** and automatically arrange in a single row by default.

Quick Example

```
<div class="container">  
  <div>1</div>  
  <div>2</div>  
  <div>3</div>  
</div>
```

Why Flexbox?

→ Align Items Effortlessly

Center content horizontally and vertically with just two properties.

→ Responsive by Nature

Flex items adapt to the available space without complex media queries.

→ Less Code

Replace float hacks and clearfix tricks with clean, readable CSS.

Flex Direction & Gap

Once you've enabled Flexbox, `flex-direction` and `gap` are two of the first properties you'll reach for. They control the axis your items flow along and the breathing room between them.

Row (Default)

Items line up **left to right** horizontally — the default Flexbox behavior.

```
.container {  
  display: flex;  
  flex-direction: row;  
}
```

Column

Items stack **top to bottom** vertically — like block elements, but with flex superpowers.

```
.container {  
  display: flex;  
  flex-direction: column;  
}
```

Gap Property

Adds consistent, equal spacing between all flex items without needing margins on individual elements.

```
.container {  
  gap: 20px;  
}
```

✅ 💡 **Pro Tip:** Using `gap` is cleaner than adding `margin` to individual items. It only adds space *between* items — never on the outer edges — keeping your layout tidy.

Justify Content & Align Items

These two properties are the heart of Flexbox alignment. Together, they give you complete control over where items sit inside their container — horizontally and vertically.

justify-content — Horizontal Axis

Distributes items along the **main axis** (left ↔ right in row direction).

Value	Effect
flex-start	Pack items to the left
center	Center all items
flex-end	Pack items to the right
space-between	Max space between items
space-around	Equal space around items

align-items — Vertical Axis

Controls item positioning along the **cross axis** (top ↔ bottom in row direction).

```
.container {  
  align-items: center;  
}
```

Complete Centering Example

```
.container {  
  display: flex;  
  justify-content: center;  
  align-items: center;  
  height: 300px;  
}
```

This is the classic pattern for perfectly centering any element — horizontally *and* vertically.

Building a Navbar with Flexbox

The navigation bar is one of the most common UI components you'll build as a front-end developer. Flexbox makes it elegant and responsive with minimal code.

HTML Structure

```
<nav>
  <h2>Logo</h2>
  <ul>
    <li>Home</li>
    <li>About</li>
    <li>Contact</li>
  </ul>
</nav>
```

CSS Styling

```
nav {
  display: flex;
  justify-content: space-between;
  align-items: center;
  background-color: black;
  color: white;
  padding: 15px;
}
ul {
  display: flex;
  gap: 20px;
  list-style: none;
}
```

Concepts Applied



Flexbox on Nav

`space-between` pushes the logo left and the links right automatically.



Gap on List

The `ul` itself becomes a flex container so nav links sit side by side with even spacing.



Vertical Centering

`align-items: center` ensures the logo and links